

簡易POSアプリ改（Lv2）テスト仕様書

対象業態：ラーメン店（店内飲食のみ・後払い会計）

項目	内容
文書名	簡易POSアプリ改（Lv2）テスト仕様書
版	v1.3
作成日	2026-09-13
作成者	おぐちゃん（Tech0 Step4 / 12期）
上位文書	要求一覧（Lv1／Lv2）、要件定義書 v1.0、設計仕様書 v1.4
ステータス	確定

改訂履歴

版	日付	内容
v0.1	2026-09-13	初版ドラフト
v1.0	2026-09-13	全体確認を実施し確定。日付変更後の再ログイン手順を追加、単価を含むリクエストの拒否を確定、UT-B-86 を追加、ST-19 を削除、設計仕様書 v1.1 との整合を反映
v1.1	2026-09-13	テスト実装可否の確認により、4.3 「テスト実装のための補足」を追加。企画重複の UT-B-87 を追加。IT-05 ・ IT-34 の実施方法を明記。設計仕様書 v1.2 との整合を反映
v1.2	2026-09-13	実装着手時の確認により改訂。AuthService は業務用 ・ トークン用の2つの Clock を受け取る。設計仕様書 v1.3 に合わせて Argon2id ・ タイムゾーン ・ Clock 分離 ・ 期間判定の一本化を反映。コード例のシングネチャと日付型を修正。IT-33 に実施条件、IT-36 ・ 37 を追加。4.3 の reducer 仕様と AuthService の注入を詳細化。ケース ID のないテストの命名規則を追加
v1.3	2026-09-14	結合テスト実施（段階9）の結果を反映。IT-34 の手順を DB タイムアウト前提に具体化。カバレッジの除外範囲（DB 接続 ・ リポジトリ）を明記。IT-19 ・ 33 の明細を明記（同日補正：改訂履歴の順序、CSP の記載を実装に合わせて補完）

1. はじめに

1.1 本書の目的

本書は、要件定義書・設計仕様書で定めた内容が実装で満たされているかを検証するためのテスト仕様書である。V字モデルの右側3工程（単体テスト・結合／機能テスト・ユーザーテスト）について、テスト計画、テスト設計、テストケースを定める。テスト実施と報告は本書の結果欄および付録Aの不具合報告書に記録する。

1.2 V字モデルとの対応

V字モデルでは、左側の各工程で決定したことを、対応する右側のテスト工程で検証する。本書の3つのテストレベルは、左側の文書と次のように対応する。



テストレベル	検証する対象	根拠となる文書・ID	観点
単体テスト (UT)	関数・クラス単位の制御フロー	設計仕様書 3.3 クラス図、6.1 計算規則・上下限	正常系、異常系、分岐網羅
結合テスト (IT)	モジュール間の連携、状態遷移、外部連携	設計仕様書 3.2 シーケンス図、5 API設計、4 データ設計	インターフェース、状態遷移、DB連携
機能テスト (ST)	要件が仕様どおり動作するか、非機能要件	要件定義書 FR-001～012、NFR	機能確認、性能、セキュリティ
ユーザーテスト (UAT)	要求が業務として満たされるか	要求一覧 REQ-01～11	業務シナリオ

単体テストで確認した観点は、上位のテストでは繰り返さない。たとえば金額計算の端数処理は単体テストで網羅し、結合テスト以上では「計算結果が API 応答と DB に正しく渡るか」だけを確認する。

1.3 テスト工程の流れ

各テストレベルは「テスト計画 → テスト設計 → テストケース作成 → テスト実施 → テスト報告」の順に進める。本書は計画・設計・ケース作成までを定め、実施と報告は結果欄と不具合報告書で行う。

2. テスト計画

2.1 テストピラミッドと配分

レベル	件数の目安	実行速度	自動化
単体テスト	全体の約70%	秒単位	pytest/jest で全自動
結合・機能テスト	約20%	分単位	API テストは自動、画面操作は手動
ユーザーテスト	約10%	時間単位	手動（店舗運用を再現）

土台となる単体テストを最も厚くし、変更のたびに全自動で回せる状態を作る。

2.2 観点と技法

テストケースは「観点（何を確かめるか）× 技法（どの値で確かめるか）」で決める。

観点	内容	主に適用するレベル
正常系	仕様どおりの入力で、期待する戻り値・状態変化になるか	UT、IT、ST、UAT
異常系	不正な入力で、例外・エラーコード・エラー表示が仕様どおりか	UT、IT、ST、UAT
制御フロー	if/switch の分岐をすべて通したか	UT
インターフェース	モジュール間でデータが正しく受け渡されるか	IT
状態遷移	画面・処理の順序が想定どおりか	IT、UAT

観点	内容	主に適用するレベル
外部連携	DB との読み書き、エラーの伝播	IT

技法	内容	本書での使い方
同値分割	同じ結果になる範囲から代表値を1つ選ぶ	通常の価格・数量・コード
境界値分析	境界とその前後を選ぶ	数量 0／1／99／100、パスワード 11／12／128／129 文字、ロック 9／10 回
デシジョンテーブル	複数条件の組み合わせを整理する	値引き適用の判定（会員×対象商品×期間）
状態遷移テスト	状態×イベントを網羅する	会計の流れ（ログイン→登録→編集→確定→リセット）
エラー推測	経験から壊れそうな入力を狙う	SQL 断片、改変した金額、同一キーの再送

組み合わせの絞り込み：正常値と異常値の全組み合わせは膨大になるため、**単一障害仮説**（異常値は1ケースに1つだけ含める）を採り、異常値同士の組み合わせは作らない。複数因子の正常値の組み合わせは、代表的な業務ケース数件に絞る。

2.3 AI と人間の役割分担

工程	担当	内容
テスト設計	人間（AI は壁打ち役）	観点の設定、因子と水準の切り分け、 期待値の決定
ケース表の作成	人間+AI	本書の表。期待値は設計仕様書の計算規則から人間が算出
テストコード作成	生成 AI	本書のケース表をもとにコード化。期待値は表の値をそのまま用いる
結果の判定	コード	pytest／jest の自動判定を最優先。画面表示は人が確認

期待値を AI に推測させない。推測させると「現在の実装＝正解」のテストになり、実装の誤りを検出できなくなる。本書ではすべての期待値を具体的な数値・文字列で明記し、AI はそれをコードに写す役割に限定する。

2.4 テスト環境

レベル	環境	構成
単体テスト	ローカル	pytest（バックエンド）、jest（フロントエンド）。DB は使わずモックで代替
結合・機能テスト	ローカル（Docker）	Next.js、FastAPI、MySQL をコンテナで起動。第3章のテストデータを投入
ユーザーテスト	Azure 検証環境	本番同等構成（Container Apps、内部 Ingress、HTTPS）。店舗の端末・カメラで実施

ユーザーテストを Azure で行う理由は、BFF の Cookie 属性（Secure）や HTTPS 終端、コールドスタートの有無が、ローカルでは再現できないためである。

2.5 ID体系と合否基準

接頭辞	レベル
UT-B-nn	単体テスト（バックエンド、pytest）

接頭辞	レベル
UT-F-nn	単体テスト（フロントエンド、jest）
IT-nn	結合テスト
ST-nn	機能テスト（非機能を含む）
UAT-nn	ユーザーテスト

合否基準

- 単体テスト：全ケース合格。カバレッジは Statements・Branch とともに 80% 以上。DB 接続と SQLAlchemy のリポジトリは単体テストでは実行できないため対象から外し、結合テストで確認する
- 結合・機能テスト：全ケース合格。不合格は不具合報告書を起票し、修正後に再実施する
- ユーザーテスト：全シナリオ完遂。重大度「高」の不具合が残存しないこと

3. テストデータ

結合・機能テストとユーザーテストで共通に使う初期データ。単体テストはこの値をもとにモックを作る。すべて架空のデータである。

3.1 担当者（staff）

staff_id	name	パスワード（平文。seed.sql には Argon2id ハッシュを記載）	状態	用途
S001	店主	ramen-owner-2026	有効	通常ログイン
S002	アルバイト A	part-timer-0001	有効	通常ログイン（2人目）
S003	退職者	retired-staff-01	無効 (is_active=FALSE)	無効化担当者の異常系
S004	ロック検証用	lock-test-user-1	有効	10回失敗によるロックの検証

3.2 会員（member）

member_id	name	phone	address	gender	age	用途
M000001	山田太郎	090-0000-0001	東京都〇〇区	男	35	通常会員
M000002	鈴木花子	NULL	NULL	女	28	電話・住所なしの会員
M999999	（未登録）	—	—	—	—	存在しない会員IDの異常系

3.3 商品（product）

product_code	name	unit_price	is_discontinued	用途
1001	醤油ラーメン	850	FALSE	通常商品
1002	味噌ラーメン	900	FALSE	通常商品
1003	冷やし中華	950	FALSE	割合値引きの対象

product_code	name	unit_price	is_discontinued	用途
1004	特製ラーメン	855	FALSE	割合値引きで端数が出る単価
2001	味玉	120	FALSE	金額値引きの対象
2002	のり	100	FALSE	金額値引きの対象
2003	メンマ	150	FALSE	金額値引きの対象
3001	餃子	400	FALSE	サイドメニュー
4001	瓶ビール	500	FALSE	ドリンク
5001	サービス品	0	FALSE	単価0円（下限）
5002	上限価格品	99999	FALSE	単価上限
1099	旧メニュー	900	TRUE	販売終了商品の異常系
9999	（未登録）	—	—	存在しないコードの異常系

3.4 税率（tax_rate）

id	tax_category	rate_bp	effective_from	用途
1	standard	1000	2019-10-01	現行税率 10%
2	standard	1200	2027-04-01	将来の税率改定（12%）。切り替えの検証

3.5 値引き企画（discount_campaign）

campaign_id	name	product_code	discount_type	discount_value	start_date	end_date
1	常連感謝トッピング企画	2001	amount	20	2026-09-01	2026-09-07
2	常連感謝トッピング企画	2002	amount	20	2026-09-01	2026-09-07
3	常連感謝トッピング企画	2003	amount	20	2026-09-01	2026-09-07
4	夏の冷やし応援	1003	percent	10	2026-07-01	2026-08-31
5	端数検証用	1004	percent	10	2026-09-01	2026-09-30
6	値引き上限検証用	2002	amount	150	2026-09-08	2026-09-30

データの初期化：ロックや取引の蓄積が後続に影響するため、各テストレベルの開始時、および S004・取引データを使うケースの前にデータを初期状態へ戻す。seed.sql の先頭で全テーブルを空にしてから投入する。

タイムゾーン：日時はすべて日本時間（設計仕様書 2.3）。TEST_FIXED_NOW の値も日本時間で指定する。

テスト実施日の基準：期間判定を含むテストは、システム日時を **2026-09-05** に固定して実施する（企画1～3・5が有効、企画4・6は期間外）。期間境界のテストでは日時を個別に指定する。日時の固定は環境変数 TEST_FIXED_NOW（設計仕様書 6.3）で行い、Azure 検証環境ではリビジョンの環境変数を更新して反映する。

4. 単体テスト

詳細設計（設計仕様書 3.3 クラス図、6.1 計算規則・上下限）に対応する。関数・クラス単位で、正常系・異常系・制御フローの分岐を確認する。DB は使わず、モックで代替する。

因子と水準の決定（人間が決定）

因子	水準	なぜこの値か
単価	0、1、850、855、99999	下限、下限直後、通常値、10%で端数が出る値（85.5）、上限
数量	1、2、99	下限、複数、上限
会員	あり、なし	値引き適用の第1条件
企画対象	対象、非対象	値引き適用の第2条件
期間	開始日前日、開始日、終了日、終了日翌日	境界の内外
値引き形式	percent、amount	要求が定める2形式
値引き値	percent 10、percent 100、amount 20、amount 150	通常、上限（明細0円）、通常、単価超過（頭打ち）
税率（bp）	0、100、1000、1200	0%、1%（改定見込み）、10%（現行）、12%（改定見込み）
課税対象額	5、999、1234	税額が $0.5 \cdot 99.9 \cdot 123.4$ となり切捨てを検証

制約条件（人間が決定）

- 期間判定は開始日・終了日を**両方含む**
- 期間判定は**日付単位**（終了日の 23:59 まで適用）
- 会員IDの先頭は**大文字 M のみ**（小文字 m は不正）
- 数量に文字列（例： "2" ）が渡された場合は**拒否**する（暗黙の型変換をしない）
- 手入力の商品コード・会員IDは、フロントで**前後の空白を trim してから**判定する。バックエンドは空白を含む値を**拒否**する（trim はフロントの責務）
- 数量は 1～99、購入リストは 1～50 行、値引き額は単価で頭打ち（設計仕様書 6.1）
- 同一商品に複数の企画が重なる場合は、**値引き額が大きい方を1つだけ適用**する（設計仕様書 6.1）

合否判定基準（人間が決定）：全ケース合格。カバレッジは Statements・Branch とともに 80% 以上。

本書にケース ID のないテスト（Clock など、カバレッジのために必要なもの）は追加してよい。テスト名を `test_extra_` で始め、本書のケースと区別する。

テスト基準日は **2026-09-05**（企画1～3・5 が有効）。期間境界のケースは日付を個別に指定する。

4.1 バックエンド（pytest）

4.1.1 PricingService.apply_discount — 値引き額の算出

規則：percent は $(\text{単価} \times \text{率}) \div 100 \times \text{数量}$ 、amount は $\min(\text{値引き額}, \text{単価}) \times \text{数量}$ 。

ID	分類	手法	入力	期待値	観点（なぜこの値か）	結果
UT-B-01	会員なし	デシジョンテーブル	会員なし、味玉120×3、企画1（amount 20）	0	会員でなければ適用しない	

ID	分類	手法	入力	期待値	観点（なぜこの値か）	結果
UT-B-02	非対象商品	デシジョンテーブル	会員、醤油850×1（企画なし）	0	対象商品でない	
UT-B-03	期間・開始前日	境界値	会員、味玉120×1、日付 8/31	0	9/1 開始の前日	
UT-B-04	期間・開始日	境界値	会員、味玉120×1、日付 9/1	20	開始日を含む	
UT-B-05	期間・終了日	境界値	会員、味玉120×1、日付 9/7 23:59	20	終了日を含む。日付単位	
UT-B-06	期間・終了翌日	境界値	会員、味玉120×1、日付 9/8 0:00	0	期間外	
UT-B-07	percent 通常	同値分割	会員、特製855×1、企画5（10%）	85	(855×10) div 100 = 85（85.5 切捨て）	
UT-B-08	percent 端数×数量	境界値	会員、特製855×3、10%	255	85×3。数量を掛けてから切り捨てると 256 になる誤りを検出	
UT-B-09	percent 上限	境界値	会員、醤油850×1、100%	850	明細が 0 円になる	
UT-B-10	amount 通常	同値分割	会員、味玉120×3、amount 20	60	20×3	
UT-B-11	amount 単価超過	境界値	会員、のり100×2、企画6（amount 150）、日付 9/10	200	min(150,100)=100、×2。負にならない	
UT-B-12	単価0・percent	境界値	会員、サービス品0×1、10%	0	(0×10) div 100 = 0	
UT-B-13	単価0・amount	境界値	会員、サービス品0×1、amount 20	0	min(20,0)=0	
UT-B-14	単価1・percent	境界値	会員、単価1×1、10%	0	(1×10) div 100 = 0	
UT-B-15	上限×上限	境界値	会員、上限価格品99999×99、10%	989901	9999×99	
UT-B-16	異常・不明な形式	エラー推測	discount_type = 'rate'	ValueError	percent／amount 以外	
UT-B-17	異常・率の範囲外	境界値	percent 0 ／ percent 101	ValueError	1～100 の外	
UT-B-87	企画の重複	デシジョンテーブル	会員、味玉120×1、企画1（amount 20）と重複期間の企画 X（percent 50）を同時に渡す	60	値引き額が大きい方（percent 50 → 60円）を1つだけ適用。合算（80）しない	

4.1.2 PricingService.calculate — 合計の算出

規則：課税対象 = 税抜合計 - 値引き合計、税額 = (課税対象 × 税率bp) div 10000、税込 = 課税対象 + 税額。期待値は（税抜合計、値引き合計、税額、税込合計）。

ID	分類	手法	入力	期待値	観点（なぜこの値か）	結果
UT-B-18	基本・非会員	同値分割	醤油850×2、税率1000	(1700, 0, 170, 1870)	1700×0.1=170	
UT-B-19	基本・会員・複数明細	同値分割	会員、醤油850×2+味玉120×3（企画1）+特製855×1（企画5）、税率1000	(2915, 145, 277, 3047)	税抜 1700+360+855。値引き60+85。課税 2770	
UT-B-20	値引き後に課税	デシジョンテーブル	会員、味玉120×3（企画1）、税率1000	(360, 60, 30, 330)	課税 300 → 税 30。値引き前に課税すると 36 になる誤りを検出（BR-02）	
UT-B-21	端数・0.5	境界値	単価5×1、税率1000	(5, 0, 0, 5)	5000 div 10000 = 0	
UT-B-22	端数・99.9	境界値	単価999×1、税率1000	(999, 0, 99, 1098)	999000 div 10000 = 99	
UT-B-23	端数・123.4	境界値	単価1234×1、税率1000	(1234, 0, 123, 1357)	1234000 div 10000 = 123	
UT-B-24	税率0%	境界値	単価1234×1、税率0	(1234, 0, 0, 1234)	税額 0	
UT-B-25	税率1%	境界値	単価1234×1、税率100	(1234, 0, 12, 1246)	123400 div 10000 = 12	
UT-B-26	税率12%	同値分割	単価1234×1、税率1200	(1234, 0, 148, 1382)	1480800 div 10000 = 148	
UT-B-27	上限	境界値	上限価格品99999×99、税率1000	(9899901, 0, 989990, 10889891)	9899901000 div 10000 = 989990	
UT-B-28	明細0円	境界値	会員、醤油850×1、percent 100、税率1000	(850, 850, 0, 0)	課税 0	
UT-B-29	異常・空リスト	エラー推測	items = []	ValueError	1件以上が必要	
UT-B-30	異常・数量0	境界値	数量 0	ValueError	下限の直前	
UT-B-31	異常・数量100	境界値	数量 100	ValueError	上限の直後	
UT-B-32	異常・負の単価	境界値	単価 -1	ValueError	下限の直前	

組み合わせの絞り込み：正常値の全組み合わせ（単価5×数量3×会員2×企画2×期間4×形式2×値4×税率4）は 15,360 通りになる。単一障害仮説により異常値は1ケースに1つとし、正常値の組み合わせは業務上の代表ケース（UT-B-19）に絞った。

4.1.3 Pydantic スキーマ検証 — 入力値の型と範囲

設計仕様書 6.1 の上下限に対応。不正な入力は 400 VALIDATION_ERROR。

ID	分類	手法	入力	期待値	観点	結果
UT-B-33	数量・下限	境界値	quantity = 1	受理	下限	

ID	分類	手法	入力	期待値	観点	結果
UT-B-34	数量・上限	境界値	quantity = 99	受理	上限	
UT-B-35	数量・下限外	境界値	quantity = 0	拒否	下限の直前	
UT-B-36	数量・上限外	境界値	quantity = 100	拒否	上限の直後	
UT-B-37	数量・負数	同値分割	quantity = -1	拒否	負の値	
UT-B-38	数量・小数	同値分割	quantity = 1.5	拒否	整数でない	
UT-B-39	数量・文字列	エラー推測	quantity = "2"	拒否	暗黙の型変換をしない (strict)	
UT-B-40	商品コード・正常	同値分割	"1001"	受理	数字のみ	
UT-B-41	商品コード・空	境界値	""	拒否	下限 (1文字) の直前	
UT-B-42	商品コード・英字	同値分割	"abc"	拒否	数字以外	
UT-B-43	商品コード・会員ID形式	エラー推測	"M0001"	拒否	会員IDを商品として送る誤り	
UT-B-44	商品コード・上限	境界値	数字20桁	受理	上限	
UT-B-45	商品コード・上限外	境界値	数字21桁	拒否	上限の直後	
UT-B-46	会員ID・下限	境界値	"M1"	受理	2文字 (M+数字1桁)	
UT-B-47	会員ID・Mのみ	境界値	"M"	拒否	数字がない	
UT-B-48	会員ID・小文字	同値分割	"m000001"	拒否	大文字 M のみ (制約)	
UT-B-49	会員ID・数字のみ	エラー推測	"1001"	拒否	商品コードを会員として送る誤り	
UT-B-50	会員ID・上限	境界値	"M"+数字19桁	受理	20文字	
UT-B-51	会員ID・上限外	境界値	"M"+数字20桁	拒否	21文字	
UT-B-52	会員ID・null	同値分割	null	受理	会員なしの正常取引	
UT-B-53	パスワード・下限外	境界値	11文字	拒否	12文字未満	
UT-B-54	パスワード・下限	境界値	12文字	受理	下限	
UT-B-55	パスワード・上限	境界値	128文字	受理	上限	
UT-B-56	パスワード・上限外	境界値	129文字	拒否	上限の直後	
UT-B-57	担当者ID・記号	同値分割	"S-001_a"	受理	英数字と - _	
UT-B-58	担当者ID・空白	同値分割	"S 001"	拒否	空白は不可	
UT-B-59	明細・空	境界値	items = []	拒否	1件以上	
UT-B-60	明細・上限	境界値	items 50件	受理	上限	

ID	分類	手法	入力	期待値	観点	結果
UT-B-61	明細・上限外	境界値	items 51件	拒否	上限の直後	
UT-B-62	冪等キー・正常	同値分割	UUID v4	受理	形式一致	
UT-B-63	冪等キー・不正	同値分割	"abc"	拒否	UUID でない	
UT-B-64	単価（明細に含む）	エラー推測	items に unit_price を含める	拒否	未知のフィールドを禁止。単価を送らない設計（7.3）	
UT-B-86	商品コード・前後空白	エラー推測	" 1001 "	拒否	trim はフロントの責務。バックエンドは厳密に検証	

4.1.4 AuthService.authenticate / refresh / revoke — 認証とロック

ID	分類	手法	入力	期待値	観点	結果
UT-B-65	正常	同値分割	S001、正しいパスワード	TokenPair 返却。failed_count = 0	認証成功	
UT-B-66	パスワード誤り	同値分割	S001、誤ったパスワード	AUTH_FAILED。failed_count = 1	失敗回数の加算	
UT-B-67	存在しないID	同値分割	X999	AUTH_FAILED	存在有無を区別しない	
UT-B-68	無効化担当者	同値分割	S003、正しいパスワード	AUTH_FAILED	無効化を存在なしと同じ応答にする	
UT-B-69	ロック直前	境界値	S004、9回連続失敗	9回目も AUTH_FAILED。locked_until = NULL	10回未満はロックしない	
UT-B-70	ロック発生	境界値	S004、10回目の失敗	locked_until = 現在 + 30分	10回でロック	
UT-B-71	ロック中・正しいPW	デシジョンテーブル	ロック中の S004、正しいパスワード	AUTH_LOCKED。failed_count 変化なし	ロック中は認証しない	
UT-B-72	ロック中・誤PW	デシジョンテーブル	ロック中の S004、誤ったパスワード	AUTH_LOCKED	同上	
UT-B-73	ロック解除直前	境界値	ロックから 29分59秒後	AUTH_LOCKED	まだロック中	
UT-B-74	ロック解除	境界値	ロックから 30分00秒後、正しいPW	成功。failed_count = 0	解除	
UT-B-75	失敗後の成功	制御フロー	3回失敗後に正しいPW	成功。failed_count = 0	成功でリセット	
UT-B-76	refresh 正常	同値分割	有効なりフレッシュトークン	新しい TokenPair。旧トークン revoked = TRUE	ローテーション	
UT-B-77	refresh 失効済み	同値分割	revoked = TRUE のトークン	TOKEN_INVALID	再利用の拒否	
UT-B-78	refresh 期限切れ	境界値	expires_at を1秒過ぎたトークン	TOKEN_INVALID	期限	

ID	分類	手法	入力	期待値	観点	結果
UT-B-79	logout	同値分割	有効なトークンで revoke	revoked = TRUE	失効	

4.1.5 TransactionService.verify_client_totals — 金額の照合

ID	分類	手法	入力	期待値	観点	結果
UT-B-80	一致	同値分割	サーバ (2915,145,277,3047)、クライアント同値	通過	正常	
UT-B-81	税抜合計ずれ	境界値	クライアントの subtotal を +1	TOTALS_MISMATCH	1円のずれを検出	
UT-B-82	値引き合計ずれ	境界値	discount_total を -1	TOTALS_MISMATCH	同上	
UT-B-83	税額ずれ	境界値	tax_amount を +1	TOTALS_MISMATCH	同上	
UT-B-84	税込合計ずれ	境界値	total を +1	TOTALS_MISMATCH	同上	
UT-B-85	全て0	エラー推測	クライアント (0,0,0,0)	TOTALS_MISMATCH	変更の典型	

pytest コード例 (UT-B-01~17 の一部)

ケース表の値をそのまま `parametrize` に写す。期待値はコード内で計算せず、表の値を書く。

```
# tests/unit/test_pricing_discount.py
from datetime import date, datetime
import pytest
from app.services.pricing import PricingService, Campaign, Item

BASE = datetime(2026, 9, 5)
C1 = Campaign(product_code="2001", discount_type="amount", discount_value=20,
               start_date=date(2026, 9, 1), end_date=date(2026, 9, 7))
C5 = Campaign(product_code="1004", discount_type="percent", discount_value=10,
               start_date=date(2026, 9, 1), end_date=date(2026, 9, 30))

@pytest.mark.parametrize("case_id, item, member, campaigns, now, expected", [
    ("UT-B-01", Item("2001", 120, 3), None, [C1], BASE, 0),
    ("UT-B-04", Item("2001", 120, 1), "M1", [C1], datetime(2026, 9, 1), 20),
    ("UT-B-05", Item("2001", 120, 1), "M1", [C1], datetime(2026, 9, 7, 23, 59), 20),
    ("UT-B-06", Item("2001", 120, 1), "M1", [C1], datetime(2026, 9, 8, 0, 0), 0),
    ("UT-B-07", Item("1004", 855, 1), "M1", [C5], BASE, 85),
    ("UT-B-08", Item("1004", 855, 3), "M1", [C5], BASE, 255),
])
def test_apply_discount(case_id, item, member, campaigns, now, expected):
    assert PricingService().apply_discount(item, member, campaigns, now) == expected

def test_UT_B_16_unknown_discount_type():
    bad = Campaign("2001", "rate", 10, date(2026, 9, 1), date(2026, 9, 7))
    with pytest.raises(ValueError):
        PricingService().apply_discount(Item("2001", 120, 1), "M1", [bad], BASE)
```

4.2 フロントエンド (jest)

4.2.1 calcTotals — 金額計算

バックエンドの `calculate` と同じ入力・同じ期待値で検証する (UT-B-18~28 に対応)。両実装が一致しなければ照合 (7.3) が成立しない。フロントは期間判定を行わないため、企画は「有効なもの」として渡す (設計仕様書 7.3)。

ID	対応	入力	期待値	結果
UT-F-01	UT-B-18	醤油850×2、税率1000	(1700, 0, 170, 1870)	
UT-F-02	UT-B-19	会員、醤油850×2+味玉120×3+特製855×1、税率1000	(2915, 145, 277, 3047)	
UT-F-03	UT-B-20	会員、味玉120×3 (企画1)、税率1000	(360, 60, 30, 330)	
UT-F-04	UT-B-21~23	単価 5/999/1234 ×1、税率1000	税額 0/99/123	
UT-F-05	UT-B-24~26	単価1234×1、税率 0/100/1200	税額 0/12/148	
UT-F-06	UT-B-27	99999×99、税率1000	(9899901, 0, 989990, 10889891)	
UT-F-07	UT-B-28	会員、850×1、percent 100	(850, 850, 0, 0)	
UT-F-08	UT-B-08	会員、855×3、percent 10	値引き 255	
UT-F-09	UT-B-11	会員、100×2、amount 150	値引き 200	

4.2.2 classifyCode — 会員ID／商品コードの判別

ID	分類	手法	入力	期待値	観点	結果
UT-F-10	会員	同値分割	"M000001"	member	先頭 M	
UT-F-11	商品	同値分割	"1001"	product	数字のみ	
UT-F-12	小文字	同値分割	"m000001"	invalid	大文字 M のみ (制約)	
UT-F-13	M のみ	境界値	"M"	invalid	数字がない	
UT-F-14	空	境界値	""	invalid	空文字	
UT-F-15	末尾 M	エラー推測	"12M"	invalid	先頭以外の M	
UT-F-16	英字混在	エラー推測	"M12a"	invalid	数字以外を含む	
UT-F-17	全角数字	エラー推測	"1 0 0 1"	invalid	全角は不可	
UT-F-18	前後の空白	エラー推測	" 1001 "	product	trim 後に判定 (手入力の空白を許容)	

4.2.3 購入リストの状態管理 (reducer)

ID	分類	手法	操作	期待値	観点	結果
UT-F-19	新規追加	同値分割	空リストに醤油を追加	1行、数量1	新規行	

ID	分類	手法	操作	期待値	観点	結果
UT-F-20	同一商品の加算	同値分割	醤油をもう1回追加	1行のまま、数量2	行を増やさない	
UT-F-21	別商品の追加	同値分割	味玉を追加	2行	新規行	
UT-F-22	上限での加算	境界値	数量99の行に追加	数量99のまま。上限フラグ ON	100にしない	
UT-F-23	選択	状態遷移	1行目を選択	選択 = 1行目	選択状態	
UT-F-24	選択の排他	状態遷移	2行目を選択	選択 = 2行目のみ	1件だけ	
UT-F-25	削除	状態遷移	選択行を削除	行が消える。選択 = なし	削除で選択解除	
UT-F-26	数量変更・下限	境界値	数量を 1 に変更	数量1	下限	
UT-F-27	数量変更・0	境界値	数量を 0 に変更	拒否（数量変わらず）	0は削除で行う	
UT-F-28	数量変更・上限	境界値	数量を 99 に変更	数量99	上限	
UT-F-29	数量変更・100	境界値	数量を 100 に変更	拒否	上限の直後	
UT-F-30	数量変更後の選択	状態遷移	数量変更	選択維持	続けて調整できる	
UT-F-31	追加後の選択	状態遷移	選択中に別商品を追加	選択解除	誤操作防止	
UT-F-32	行数上限	境界値	50種類目を追加	50行	上限	
UT-F-33	行数上限外	境界値	51種類目を追加	拒否	上限の直後	
UT-F-34	リセット	状態遷移	確定後のリセット	リスト空、会員 null、選択 null	次の客へ	

4.2.4 画面コンポーネント表示

ID	分類	手法	条件	期待値（表示される文言・要素）	結果
UT-F-35	担当者表示	同値分割	ログイン中 S001（店主）	「店主」が表示される	
UT-F-36	未登録商品	同値分割	商品照会が 404	「商品がマスタ未登録です」	
UT-F-37	未登録会員	同値分割	会員照会が 404	「該当する会員が存在しません」	
UT-F-38	上限到達	境界値	数量99で追加	「上限に達しています」	
UT-F-39	値引き額	同値分割	味玉×3 に企画1	行に「-60」が表示される	
UT-F-40	会員なし	同値分割	会員未入力	会員欄が「会員なし」表示、値引きなし	
UT-F-41	電話・住所非表示	エラー推測	会員 M000001 を読込	画面に電話番号・住所が含まれない	

jest コード例（UT-F-01～03）

```
// __tests__/calcTotals.test.ts
import { calcTotals } from "@lib/pricing";

// GET /settings が返す「本日より有効な企画」をそのまま渡す。フロントは期間判定をしない
const C1 = { campaign_id: 1, name: "常連感謝", product_code: "2001", discount_type: "amount", di
```

```

scount_value: 20 } as const;
const C5 = { campaign_id: 5, name: "端数検証", product_code: "1004", discount_type: "percent", discount_value: 10 } as const;

describe("calcTotals", () => {
  it("UT-F-01 非会員・醤油×2 → (1700, 0, 170, 1870)", () => {
    const r = calcTotals([
      { code: "1001", name: "醤油ラーメン", unitPrice: 850, qty: 2 },
    ], null, 1000);
    expect(r).toEqual({ subtotal: 1700, discountTotal: 0, taxAmount: 170, total: 1870 });
  });
  it("UT-F-02 会員・複数明細 → (2915, 145, 277, 3047)", () => {
    const items = [
      { code: "1001", name: "醤油ラーメン", unitPrice: 850, qty: 2 },
      { code: "2001", name: "味玉", unitPrice: 120, qty: 3 },
      { code: "1004", name: "特製ラーメン", unitPrice: 855, qty: 1 },
    ];
    const r = calcTotals(items, "M000001", [C1, C5], 1000);
    expect(r).toEqual({ subtotal: 2915, discountTotal: 145, taxAmount: 277, total: 3047 });
  });
  it("UT-F-03 値引き後に課税 → 税額 30 (36 ではない)", () => {
    const r = calcTotals([
      { code: "2001", name: "味玉", unitPrice: 120, qty: 3 },
    ], "M000001", [C1], 1000);
    expect(r.taxAmount).toBe(30);
  });
});

```

実行: `npm test -- --coverage` (フロント)、`pytest --cov=app --cov-branch` (バック)。カバレッジレポートの Statements・Branch を合否判定に用いる。

4.3 テスト実装のための補足

テストコードを生成 AI に書かせる際に必要な、関数の形と差し替え点を定める。設計仕様書 3.3 クラス図・5.2 型定義に対応する。

バックエンド：テスト対象の関数と型

対象	シグネチャ	戻り値／例外
<code>PricingService.apply_discount</code>	<code>(item: Item, member: str \ None, campaigns: list[Campaign], now: datetime) -> int</code>	明細の値引き額 (数量分の合計)。不正な形式・率は <code>ValueError</code>
<code>PricingService.calculate</code>	<code>(items: list[Item], member: str \ None, campaigns: list[Campaign], tax_rate_bp: int, now: datetime) -> Totals</code>	<code>Totals(subtotal, discount_total, tax_amount, total)</code> 。空リスト・数量範囲外・負の単価は <code>ValueError</code>

対象	シグネチャ	戻り値／例外
TransactionService.verify_client_totals	(server: Totals, client: Totals) -> None	不一致なら TotalsMismatch 例外 (server_totals を保持)
AuthService.authenticate	(staff_id: str, password: str) -> TokenPair	失敗は AuthError(code="AUTH_FAILED" \ "AUTH_LOCKED")
AuthService.refresh	(refresh_token: str) -> TokenPair	失効・期限切れは AuthError(code="TOKEN_INVALID")
AuthService.revoke	(refresh_token: str) -> None	—
Item	product_code: str, unit_price: int, quantity: int	dataclass
Campaign	product_code, discount_type, discount_value, start_date, end_date	dataclass。日付は date

差し替え点： AuthService(staff_repo: StaffRepository, token_repo: TokenRepository, business_clock: Clock, token_clock: Clock, settings: AuthSettings) の形で注入する。ロック判定 (UT-B-69～74) は business_clock、JWT とリフレッシュトークンの期限 (UT-B-76～78) は token_clock で判定する。settings は JWT 署名鍵と有効期間を持つ。単体テストでは2つのリポジトリをインメモリの偽実装、clock を任意の日時を返す偽実装に差し替える。UT-B-73／74 (29分59秒／30分00秒) は clock を進めて検証する。ロック発生時に failed_count は0に戻る (設計仕様書 7.1)。PricingService は DB を参照しないため差し替え不要。

Pydantic スキーマ (設計仕様書 5.2 の TypeScript 型と同名)：LoginRequest、TransactionRequest、TransactionItem、ClientTotals。パスパラメータの商品コード・会員IDは validate_product_code(str)／validate_member_id(str) の関数で検証し、これを UT-B-40～51・86 の対象とする。すべてのスキーマは extra="forbid"。

フロントエンド：テスト対象の関数と reducer

対象	シグネチャ
calcTotals	(items: CartLine[], memberId: string \ null, campaigns: Campaign[], taxRateBp: number) => Totals。campaigns は GET /settings が返す有効な企画。期間判定はしない
classifyCode	(raw: string) => "member" \ "product" \ "invalid"。内部で trim してから判定。長さ上限 (20文字) も確認する
cartReducer	(state: CartState, action: CartAction) => CartState

CartLine : { code: string, name: string, unitPrice: number, qty: number }

CartState : { lines: CartLine[], selectedCode: string \ | null, memberId: string \ | null, error: "QUANTITY_LIMIT" \ | "LINE_LIMIT" \ | null }。error は次に成功した操作で null に戻す

CartAction :

アクション	内容	対応ケース
ADD_PRODUCT { product }	追加または数量+1。99なら error = "QUANTITY_LIMIT"。50行のとき新しい商品なら error = "LINE_LIMIT"（既存商品の +1 は許可）。選択は解除	UT-F-19～ 22、31～33
SELECT_LINE { code }	選択（排他）	UT-F-23～ 24
REMOVE_SELECTED	選択行を削除し、選択解除。未選択なら何もしない	UT-F-25
SET_QUANTITY { qty }	1～99 のみ受理。範囲外は state を変えない。未選択なら何もしない。選択は維持	UT-F-26～ 30
SET_MEMBER { memberId }	会員を設定（null で会員なし）	—
RESET	初期状態に戻す	UT-F-34

テストデータの投入

第3章のデータは `tests/fixtures/seed.sql` として用意する。担当者のパスワードは 3.1 の平文から事前に計算した Argon2id ハッシュを記載する。テーブルは `schema.sql`（管理者権限で実行）で作成し、`seed.sql` は先頭で全テーブルを空にしてから投入する。各テストレベルの開始時に `seed.sql` を再投入して初期化する。

5. 結合・機能テスト

結合テストは基本設計（設計仕様書の API 設計・シーケンス図・データ設計）に、機能テストは要件定義書（FR・NFR）に対応する。環境は Docker（Next.js・FastAPI・MySQL）で、第3章のテストデータを投入した状態で実施する。

単体テストで確認済みの観点は含めない。金額計算の正しさは第4章で網羅済みのため、本章では計算結果が API 応答と DB に正しく渡ることだけを確認する。

判定方法（人間が決定）

- 性能：10回測定し、9回以上が目標値以内であれば合格
- セキュリティ：設計仕様書 7 章の9項目すべてを対象とする

5.1 結合テスト

5.1.1 インターフェース — API の入出力

設計仕様書 5.2 の型定義どおりに入出力が行われるか。BFF（Next.js）経由で呼び出す。

ID	API	分類	入力	期待値	観点	結果
IT-01	1 login	正常	S001、正しいパスワード	200。ボディは {staff_id, name} のみ。Set-Cookie でアクセス・リフレッシュの2トークン (httpOnly)	トークンをボディに含めない	
IT-02	1 login	異常	誤ったパスワード	401 AUTH_FAILED。ボディにスタックトレースを含まない	エラー形式	
IT-03	1 login	異常	S003（無効化）	401 AUTH_FAILED（存在しない場合と同一応答）	存在有無を漏らさない	
IT-04	1 login	異常	S004 を10回失敗させた後	423 AUTH_LOCKED	ロックの伝播	
IT-05	2 refresh	正常	ACCESS_TOKEN_TTL_SECONDS=5 で起動し、ログイン後6秒待ってから商品照会	BFF が自動更新し、200 で商品が返る。ブラウザには 401 が届かない	透過的な更新	
IT-06	3 logout	正常	ログアウト後に商品照会	401 TOKEN_INVALID。ログイン画面へ	失効	
IT-07	4 settings	正常	基準日 9/5	tax_rate_bp: 1000（将来日付の 1200 を返さない）、campaigns に企画1〜3・5のみ（4・6 は含まない）	有効な税率・企画だけを返す	
IT-08	5 members	正常	M000001	200 {member_id, name}。phone・address を含まない	表示範囲の限定	
IT-09	5 members	正常	M000002（電話・住所 NULL）	200。エラーにならない	NULL 許容	
IT-10	5 members	異常	M999999	404 MEMBER_NOT_FOUND	未登録	
IT-11	6 products	正常	1001	200 {product_code, name: "醤油ラーメン", unit_price: 850}	照会	
IT-12	6 products	異常	9999	404 PRODUCT_NOT_FOUND	未登録	
IT-13	6 products	異常	1099（販売終了）	404 PRODUCT_NOT_FOUND	廃止商品は未登録と同じ扱い	
IT-14	7 transactions	正常	UT-B-19 と同じ明細、client_totals (2915,145,277,3047)	201。応答の totals が一致。lines が3行で値引き額 0／60／85	計算結果の受け渡し	

ID	API	分類	入力	期待値	観点	結果
IT-15	7 transactions	異常	client_totals の total を 3048	409 TOTALS_MISMATCH。details.server_totals に (2915,145,277,3047)	照合の伝播	
IT-16	7 transactions	異常	同じ idempotency_key で2回送信	2回目は 409 DUPLICATE。details.transaction_id が1回目と同じ。DB の取引は1件	冪等性	
IT-17	7 transactions	異常	items = []	400 VALIDATION_ERROR	空リスト	
IT-18	7 transactions	異常	商品コード 9999 を含む	404 PRODUCT_NOT_FOUND。取引は保存されない	部分保存しない	
IT-19	7 transactions	正常	member_id = null、明細は UT-B-18 (醤油850×2)	201。値引き 0。DB の member_id が NULL	会員なし取引	
IT-20	共通	異常	未知のエンドポイント /api/xxx	404。スタックトレースなし	エラー形式	
IT-36	7 transactions	異常	member_id = M999999 (形式は正しいが未登録)	404 MEMBER_NOT_FOUND。取引は保存されない	外部キー違反を 500 にしない	
IT-37	7 transactions	異常	items に商品コード 1001 が2行ある	400 VALIDATION_ERROR	フロントが集約済みの前提を検証	

5.1.2 状態遷移 — 会計の流れ

設計仕様書 3.1 アクティビティ図の遷移を、ブラウザ操作で確認する。

ID	分類	手法	操作の順序	期待値	結果
IT-21	基本の流れ	状態遷移	ログイン → 会員読込 → 商品2点 → 購入 → ポップアップを閉じる	各画面が順に遷移し、最後にリスト・会員欄が空になる	
IT-22	会員が後	状態遷移	ログイン → 商品 (企画対象) → 会員読込 → 購入	会員読込の時点で値引き額が表示され、購入時の金額に反映される	
IT-23	会員なし	状態遷移	ログイン → 商品 → 「会員なし」 → 購入	エラーにならず確定。値引きなし	
IT-24	編集を挟む	状態遷移	商品3点 → 2点目を選択 → 削除 → 1点目の数量を変更 → 購入	合計が編集後の内容で確定する	
IT-25	空で購入	状態遷移	ログイン → 何も登録せず購入	確定しない。リストは空のまま	
IT-26	連続会計	状態遷移	1件確定 → 閉じる → 次の会員読込 → 商品 → 購入	2件目が1件目の内容を引き継がずに確定する	

ID	分類	手法	操作の順序	期待値	結果
IT-27	未登録を挟む	状態 遷移	商品 → 未登録コード → 商品	未登録のエラー後も、リストは保持され 続行できる	
IT-28	ログイン失敗 から復帰	状態 遷移	誤パスワード → 正しいパスワード	2回目でレジ画面へ遷移	

5.1.3 外部連携 — DB への保存

ID	分類	確認内容	期待値	結果
IT-29	スナッ プショ ット	IT-14 の取引の transaction_detail を参照	product_name ・ unit_price ・ discount_amount ・ tax_rate_bp が購入時点 の値で保存されている	
IT-30	マスタ 変更後 の履歴	IT-14 の後、醤油ラーメンの単価を 950 に変更 し、IT-14 の取引を再参照	明細の unit_price は 850 のまま	
IT-31	税率変 更後の 履歴	税率マスタに「翌日から 1200」を追加し、IT-14 の取引を再参照	tax_rate_bp は 1000 のまま	
IT-32	担当者 の記録	S002 でログインして確定	transaction.staff_id = S002	
IT-33	日時の 記録	TEST_FIXED_NOW を未設定にして起動し、UT- B-18 の明細（値引きなし）で確定直後に参照	transacted_at が実時刻と一致（±5秒）	
IT-34	トラン ザクシ ョン	docker pause mysql → 購入ボタン → 10秒 以上待って応答を確認 → docker unpause mysql → 12秒待って DB を確認 → 同じ内容で 再度購入	1回目は 500（約10秒後）で「処理に失敗し ました」。unpause 後も取引は保存されな い。2回目は 201 で取引が1件だけ保存される	
IT-35	更新・ 削除 API の不在	取引の PUT／DELETE を送る	405 または 404。取引は変化しない	

5.2 機能テスト（FR-001～012）

要件定義書 9.2 のテスト観点をケース化する。単体・結合で確認済みの細部は含めない。

ID	要件	分類	操作・入力	期待値	結果
ST-01	FR-001	正常	担当者ID・パスワードでログイン	レジ画面に遷移し、担当者名が表示される	
ST-02	FR-001	異常	誤った組み合わせ	「担当者IDまたはパスワードが正しくありません」。レジ画面に遷移しない	
ST-03	FR-002	正常	会員証をスキャン	会員名と「割引対象」が表示される	
ST-04	FR-002	正常	会員IDを手入力	スキャンと同じ結果	

ID	要件	分類	操作・入力	期待値	結果
ST-05	FR-002	異常	未登録の会員ID	「該当する会員が存在しません」。続行できる	
ST-06	FR-003	正常	早見表をスキャン	商品名・単価が表示され「1件追加されました」	
ST-07	FR-003	正常	3商品を連続スキャン	3行が追加される。途中でスキャンが止まらない	
ST-08	FR-004	正常	商品コードを手入力して追加	スキャンと同じ挙動	
ST-09	FR-005	異常	未登録コード（スキャン・手入力の両方）	「商品がマスタ未登録です」。リストに追加されない	
ST-10	FR-006	正常	同一商品を2回登録	1行で数量2。入力欄がクリアされる	
ST-11	FR-006	境界	数量99の行に再登録	数量99のまま。「上限に達しています」	
ST-12	FR-007	正常	行を選択	選択行が強調表示され、名称・単価・数量が表示される	
ST-13	FR-007	正常	選択行を削除	行が消え、合計が再計算される	
ST-14	FR-007	境界	数量を 1/99 に変更	変更でき、合計に反映	
ST-15	FR-007	異常	数量を 0/100 に変更	拒否される	
ST-16	FR-008	正常	会員あり、味玉×3（企画1）	行に「-60」。合計に反映	
ST-17	FR-008	正常	会員あり、特製×1（企画5、10%）	行に「-85」	
ST-18	FR-008	異常	会員なしで企画対象商品	値引きなし。エラーにならない	
ST-20	FR-009	正常	UT-B-19 の明細	税抜 2,915・値引き 145・税込 3,047 が表示される	
ST-21	FR-010	正常	購入ボタン	ポップアップに税込合計。DB に取引が保存される	
ST-22	FR-010	異常	空リストで購入	確定しない	
ST-23	FR-011	正常	ポップアップを閉じる	入力欄・リスト・会員欄がすべて空になり、会員読込から再開できる	
ST-24	FR-012	正常	税率マスタに 1200（翌日から）を追加し、日付を進めて 再ログイン 後に会計	税額が12%で計算される	
ST-25	FR-012	正常	ST-24 の後、変更前の取引を参照	金額が変わっていない	

5.3 非機能テスト

性能（NFR-PERF）

ID	要件	測定内容	判定	環境	結果
ST-26	PERF-01	スキャンから商品名・単価表示まで	10回中9回以上が1秒以内	Docker	
ST-27	PERF-02	会員読込→商品3点→購入確定	10回中9回以上が1分以内	Docker	
ST-28	PERF-03	3時間放置後の初回スキャン	10回中9回以上が1秒以内（放置前と同水準）	Azure	
ST-29	PERF-04	取引 5万件（1年分相当）投入後の ST-26	同上	Docker	

セキュリティ（NFR-SEC）

ID	要件	分類	方法	期待値	環境	結果
ST-30	SEC-01	異常	Cookie なしで /api/products/1001	401 TOKEN_INVALID。商品情報が返らない	Docker	
ST-31	SEC-07	異常	購入確定の client_totals を改変（total を -1000）	409。取引は保存されない	Docker	
ST-32	SEC-07	異常	items に unit_price: 1 を含めて送信	400 VALIDATION_ERROR。取引は保存されない	Docker	
ST-33	SEC-08	異常	商品コードに 1001' OR '1'='1	400。DB エラーにならない	Docker	
ST-34	SEC-08	異常	会員IDに M1; DROP TABLE member; --	400。テーブルは残る	Docker	
ST-35	SEC-09	異常	会員照会の応答を確認	phone・address が含まれない	Docker	
ST-36	SEC-06	異常	ログアウト後のリフレッシュトークンで refresh	401	Docker	
ST-37	SEC-06	正常	ログイン応答の Set-Cookie	httpOnly・Secure・SameSite=Strict が付与	Azure	
ST-38	SEC-05	異常	存在しないIDと誤パスワードの応答を比較	文言・ステータスが同一	Docker	
ST-39	7.4	異常	本番設定で /docs、/redoc、/openapi.json	すべて 404	Azure	
ST-40	7.2	異常	外部から FastAPI の URL に直接アクセス	到達不可（タイムアウトまたは拒否）	Azure	
ST-41	SEC-10	確認	リポジトリを検索	DB 接続文字列・JWT 署名鍵が含まれない	—	

6. ユーザーテスト

要求一覧（REQ-01～11）に対応する。レジ担当者が客に対応する1回の会計を業務シナリオとして通して実施し、要求が業務として満たされているかを確認する。

実施体制（人間が決定）

項目	内容
環境	Azure 検証環境（本番同等構成）。店舗で使用する端末とカメラ、印刷したメニュー早見表・会員証を使う
実施者	店主役1名、アルバイト役1名。アルバイト役は利用者マニュアルだけを見て操作する（説明なしで使えるかの確認を兼ねる）
時期	営業時間外
判定	各シナリオを完遂できれば合格。操作に迷った箇所は不具合ではなく「改善要望」として別に記録する
通信断の再現	端末の Wi-Fi をオフにする（店舗で実際に起きるのは端末側の回線切れであるため）

シナリオの前提：第3章のテストデータを投入し、システム日付は 2026-09-05（企画1～3・5 が有効）とする。日付を変えるシナリオは個別に指定する。税率と値引き企画はログイン時に取得されるため（設計仕様書 3.2.2）、**日付を変えた後は必ず再ログインする。**

6.1 正常系

ID	場面	要求	手順	期待値	結果
UAT-01	開店準備	REQ-01	アルバイト役が S002 でログインする	レジ画面に「アルバイトA」が表示される	
UAT-02	一般客の会計	REQ-03～07、09	会員証なし。伝票「醤油ラーメン1」を見て早見表をスキャン→会員なしを選択→購入→ポップアップを閉じる	税抜 850・税込 935 が表示され、確定後に画面が空になる。DB に取引1件（member_id NULL、staff_id S002）	
UAT-03	常連客の会計	REQ-02、11	会員証 M000001 をスキャン→味噌ラーメン1・味玉1 をスキャン→購入	会員名「山田太郎」表示。味玉の行に「-20」。税抜 1,020・値引き 20・税込 1,100	
UAT-04	会員証が後から出てくる	REQ-02、11	味玉1 をスキャン→客が「カード持ってた」→会員証をスキャン→購入	会員証を読んだ時点で味玉の行に「-20」が現れる。UAT-03 と同じ金額で確定	
UAT-05	グループ会計（8人）	REQ-06、09、10	テーブル2卓分の伝票1枚：醤油3・味噌3・冷やし中華2・餃子4・瓶ビール5。伝票の順に早見表をスキャン→購入	同一商品は1行に数量集約され、5行になる。税抜 8,900・税込 9,790。確定できる	
UAT-06	申告による修正	REQ-10	UAT-05 の途中で「ビール1本は隣の席の分」→ビールの行を選択→数量を 5→4 に変更→「餃子は頼んでない」→餃子の行を選択→削除→購入	税抜 6,800・税込 7,480 で確定	
UAT-07	早見表が読めない	REQ-03	汚損したバーコード（意図的に一部を塗りつぶした早見表）をスキャン→読めないので商品コード 1001 を手入力→追加	スキャンと同じく醤油ラーメンが登録される	

ID	場面	要求	手順	期待値	結果
UAT-08	連続会計	REQ-06	3組が続けてレジへ：①醤油1（会員なし） ②味噌1（M000001）③餃子1（会員なし）。各組を確定して閉じる	各組の内容が前の組を引き継がない。DBに3件、それぞれ正しい内容	
UAT-09	履歴の確認	REQ-07	閉店後、店主役がDBで当日の取引を参照する	いつ・何を・いくつ・いくらで・誰が買って（会員ID）・誰が処理した（担当者ID）かが揃っている	
UAT-10	税率改定	REQ-08	店主役が税率マスタに「2026-09-06 から1200」を登録→システム日付を9/6に進める→ 再ログイン →醤油1を会計→9/5の取引を再参照	9/6の会計は税込 952（850×1.12=952）。9/5の取引は税込 935のまま	

6.2 異常系

ID	場面	要求	手順	期待値	結果
UAT-11	未登録商品	REQ-03、09	新メニュー（コード 9999、マスタ未登録）の早見表をスキャン→エラー→店主に連絡する想定で、他の商品（醤油1）を続けて登録→購入	「商品がマスタ未登録です」が表示され、醤油だけで確定できる	
UAT-12	会員証の読み取り不能	REQ-02	汚損した会員証をスキャン→読めないので会員ID M000001を手入力→味玉1→購入	値引きが適用される	
UAT-13	通信断	NFR-OPS-05、06	味玉1を登録→端末のWi-Fiをオフ→購入ボタン→エラー表示→Wi-Fiをオン→もう一度購入ボタン	1回目は「通信できません」でリストが保持される。2回目で確定し、DBの取引は1件のみ。復旧しない場合は手書き伝票で会計する運用へ切り替える	
UAT-14	ログインロック	NFR-SEC-04	アルバイト役がS004で誤パスワードを10回入力→11回目に正しいパスワード→店主役がDBでlocked_untilをNULLにする→再ログイン	11回目は「一定時間後に再試行してください」。解除後にログインできる	

6.3 境界値

ID	場面	要求	手順	期待値	結果
UAT-15	最小の会計	REQ-04、05	味玉1のみ（会員なし）→購入	税抜 120・税込 132。1品でも確定できる	
UAT-16	大量注文	REQ-09、10	団体客で醤油ラーメン10杯。早見表を10回スキャン	1行で数量10。税抜 8,500・税込 9,350	
UAT-17	値引き期間の最終日	REQ-11	システム日時を 2026-09-07 21:59 に設定→再ログイン→M000001→味玉1→購入	値引き「-20」が付く（終了日を含む）	
UAT-18	値引き期間の翌日	REQ-11	システム日時を 2026-09-08 11:00 に設定→再ログイン→M000001→味玉1→購入	値引きなし	

期待値の算出根拠（税率10%、切捨て）

シナリオ	明細	税抜	値引き	課税対象	税額	税込
UAT-02	850	850	0	850	85	935
UAT-03	900+120	1,020	20	1,000	100	1,100
UAT-05	850×3+900×3+950×2+400×4+500×5	8,900	0	8,900	890	9,790
UAT-06	UAT-05 -ビール1 -餃子4	6,800	0	6,800	680	7,480
UAT-10 (9/6)	850、税率12%	850	0	850	102	952
UAT-15	120	120	0	120	12	132
UAT-16	850×10	8,500	0	8,500	850	9,350

UAT-05 は非会員のため、冷やし中華の企画4（8/31 終了）は日付に関わらず適用されない。

7. トレーサビリティマトリクス

要求 → 要件 → 設計要素 → テストケースの4段で追跡できることを示す。要求一覧の各項目が、どのテストレベルのどのケースで検証されるかを1行で確認できる。

7.1 要求からテストへ

要求	要件 (FR)	設計要素	単体 (UT)	結合 (IT)	機能 (ST)	ユーザー (UAT)
REQ-01 ログイン	FR-001	3.2.1、API 1～3、7.1	B-65～79	01～06、28	01～02	01、14
REQ-02 会員カード	FR-002	API 5、6.1 コード体系	B-46～52、F-10～18	08～10	03～05	03、04、12
REQ-03 商品コード手入力	FR-004～006	API 6、3.1	B-40～45	11～13	08～09	07
REQ-04 税抜・税込表示	FR-009	6.1 計算規則	B-18～28、F-01～07	14	20	02、15
REQ-05 購入確定	FR-010	3.2.3、API 7	B-80～85	14～19、36～37	21～22	02、13
REQ-06 リセット・再開	FR-011	3.1	F-34	21、26	23	08
REQ-07 履歴保存	FR-010	4.2 transaction、4.1 スナップショット	—	29～35	21	09
REQ-08 税率可変	FR-009、FR-012	4.2 tax_rate	B-24～26、F-05	31	24～25	10
REQ-09 バーコード登録	FR-003、005、006	3.2.2、API 6	F-19～22	11～13、27	06～07、10～11	02、05、11、16
REQ-10 選択・削除・数量変更	FR-007	3.1、7章②	F-23～33	24	12～15	05、06
REQ-11 会員特典値引き	FR-008	6.1、7.3	B-01～17、B-87、F-08～09	07、14、22	16～18	03、04、17、18

充足の確認：REQ-01～11 のすべてに、単体・結合・機能・ユーザーの各レベルで少なくとも1つのテストケースが対応する（REQ-07 は単体テストの対象となる関数がないため、結合以上で検証する）。

7.2 非機能要件からテストへ

要件	設計要素	テスト
NFR-PERF-01～04	2.3、3.2.2	ST-26～29
NFR-OPS-01	2.3 Container Apps 常時稼働	ST-28（コールドスタートが起きないこと）
NFR-OPS-02～03	4.2、3.2.3	IT-29～34、UAT-09
NFR-OPS-04	5.1（更新・削除 API なし）	IT-35
NFR-OPS-05	4.2 idempotency_key、6.2	IT-16、UAT-13
NFR-OPS-06	運用	UAT-13（手書き運用への切り替え）
NFR-OPS-07～08	運用	テスト対象外（運用ルールの合意で確認）
NFR-SEC-01	7.1	ST-30
NFR-SEC-02～05	6.1、6.2、7.1	B-53～56、B-65～75、ST-38、UAT-14
NFR-SEC-06	7.1 Cookie、refresh_token	B-76～79、IT-05～06、ST-36～37
NFR-SEC-07	7.3	B-80～85、IT-15、ST-31～32
NFR-SEC-08	6.1、7.5	B-33～64、B-86、ST-33～34
NFR-SEC-09	5.2 API 5	F-41、IT-08、ST-35
NFR-SEC-10	2.3	ST-41
NFR-MNT-01	4.2 tax_rate、discount_campaign	IT-31、ST-24（マスタ変更のみで反映）
NFR-MNT-02～04	2.2、2.1、7.6	テスト対象外（設計レビュー・成果物確認）
7.2 BFF／7.4 Swagger	2.1、7.4	ST-39～40

7.3 件数のまとめ

レベル	件数	構成比
単体テスト	128（B 87、F 41）	約64%
結合テスト	37	約18%
機能テスト	40（機能24、非機能16。ST-19 は重複のため欠番）	約20%
ユーザーテスト	18	約9%
合計	223	

テストピラミッド（単体70：統合20：E2E 10）にほぼ沿った構成である。

付録A. 不具合報告書テンプレート

テスト実施で不合格となったケースは、以下の形式で記録する。1件1報告。

項目	記入内容
報告ID	BUG-nnn

項目	記入内容
発見日	yyyy-mm-dd
発見者	
テストケースID	不合格となったケース（例：IT-15）
環境	Docker／Azure、ブラウザ、端末
再現手順	番号付きで、誰がやっても同じ結果になる粒度で
期待結果	テスト仕様書の期待値をそのまま転記
実際の結果	画面表示・応答・DBの状態。スクリーンショットや応答 JSON を添付
重大度	高：業務が止まる／金額が誤る、中：代替手段で継続できる、低：表示・文言
原因（判明後）	要件・設計・実装のどの工程の誤りか
対応	修正内容、修正したファイル、再テストの結果
ステータス	起票／調査中／修正済み／再テスト合格／保留

原因の工程を記録する理由：要件や設計の誤りが実装で見つかった場合、修正は要件定義書・設計仕様書に遡って行う（一方通行の原則）。実装だけを直すと、文書と実装が食い違ったまま残る。

付録B. 生成AI活用の記録

本書は Claude（doc-coauthoring スキル）との対話により作成した。講義資料「テスト工程で AI に任せる範囲／人間が決める範囲」に従い、**テスト設計は人間が行い、AI は壁打ち役**とした。

人間が決定した事項

工程	決定内容
テストの目的・観点	単体テストの対象9関数、各レベルの観点、レベル間で観点を重複させない方針
因子と水準	単価5水準、数量3水準、税率4水準（0・1・10・12%。1%と0%は改定見込みとして人間が追加）など
制約条件	期間判定は開始日・終了日を含む、日付単位。会員IDは大文字 M のみ。数量の文字列は拒否。手入力の前後空白はフロントで trim、バックエンドは拒否。単価を含むリクエストは拒否。企画の重複は値引き額が大きい方を適用。日時固定は環境変数で行う
期待値	値引き17件・合計15件の全期待値を人間が検算し承認。 C-02（税込 3,047）について「3,206 では」との指摘があり、値引き適用の有無を確認して 3,047 で確定した
合否判定基準	カバレッジ 80% 以上。性能は10回中9回以上。セキュリティは設計仕様書7章の全項目
ユーザーテスト	シナリオの過不足（グループ会計を8人に変更）、実施体制、通信断の再現方法

AI が担った事項

- 講義資料に沿った章立てと、観点×技法の表の構成
- 因子・水準・期待値の**案**の作成（決定は人間）
- 計算規則からの期待値の算出と算出過程の提示
- 223件のテストケース表への展開、ID付与、トレーサビリティ表の作成
- pytest／jest のコード例（ケース表の値を写す形）

学び

期待値の検算で、人間と AI の計算が食い違う場面があった。C-02 で人間側が「3,206」と算出したのは値引きを引き忘れたためだったが、AI が算出過程を示して確認したことで、人間が計算規則（値引き後に課税）を再確認し、3,047 を正として承認した。期待値を AI に任せきりにせず、算出過程を人間が追える形で提示することが、「現在の実装＝正解」のテストを防ぐ実務上の手段になる。

テスト設計が設計書の齟齬を見つけた。単体テスト UT-B-01（会員なしなら値引き 0）を書いた時点で、設計仕様書のクラス図にある `apply_discount(item, campaigns, now)` には会員の引数がなく、会員判定ができないことが判明した。設計仕様書を v1.1 に改訂し、引数に `member` を追加した。

制約条件は設計書に書かれていないものが残っていた。期間判定が終了日を含むか、日付単位か日時単位か、手入力の空白を許容するか。いずれも設計仕様書には明記されておらず、テストケースの期待値を決める段階で初めて判断が必要になった。テスト設計は、上流の文書の抜けを最後に洗い出す工程でもある。

AI に「なぜこの値か」を書かせると、水準の妥当性を人間が判断できる。単価 855 円を水準に入れた理由（10%引きで 85.5 円になり、端数処理の位置で結果が変わる）を表に書いたことで、人間が「この値は必要」と判断する材料になった。値だけを並べた表では判断できない。

テストコードは生成 AI が書く前提で、ケース表を「そのまま写せる」形にした。ID・入力・期待値を表に明記し、コード例では `parametrize` に表の値を写すだけにしている。AI が期待値を推測する余地を残さないための構成である。